🛡️ Trazabilidad con Middleware

¿Qué es el Middleware de Trazabilidad?

Un **Middleware** es un componente que se sitúa entre la recepción de una petición HTTP y su controlador final en una aplicación web (por ejemplo, en FastAPI).

Cada vez que un cliente realiza una consulta, el middleware registra automáticamente:

- Marca de tiempo de la petición.
- Método y ruta de acceso (ej: `POST /api/chat`).
- Tiempo total de procesamiento.
- Atributos del cuerpo de la petición (captura de *inputs*).

El uso de un middleware es una excelente práctica porque evita modificar el código de los endpoints individuales; la captura de datos ocurre de forma centralizada y escalable.

🚀 Ejemplo de Implementación en una API

A continuación, se detalla cómo crear el middleware (`GenAITrackingSSEMiddleware`) y cómo registrarlo en la aplicación principal (`main.py`).

1. Código del Middleware

Crea un archivo en el proyecto (por ejemplo, `genai_sse_middleware.py`). Este componente abstrae el inicio y fin de un Span de OpenTelemetry, soportando respuestas estándar y flujos de datos (*Streaming/SSE*).

```python
# app/presentation/api/middleware/genai_sse_middleware.py

import json
from typing import Callable, Dict, Optional, Any
from fastapi import Request
from starlette.middleware.base import BaseHTTPMiddleware
from starlette.responses import StreamingResponse, Response
from opentelemetry.context import attach, detach
from opentelemetry.trace import Status, StatusCode, set_span_in_context

# Importamos el módulo de inicialización
from app import telemetry_singleton as telemetry

class GenAITrackingSSEMiddleware(BaseHTTPMiddleware):
    """
    Middleware que intercepta peticiones HTTP y registra la trazabilidad.
    Soporta operaciones GenAI y respuestas StreamingResponse (SSE).
    """
    def __init__(
        self,
        app,
```

```
        tracer,
        server_host: str,
        service_type: str,
        operation_map: Optional[Dict[str, str]] = None,
        capture_inputs: bool = True,
        capture_input_max_len: int = 2000,
        enabled_paths: Optional[set[str]] = None,
        carrier_mode: str = "fallback",
    ):
        super().__init__(app)
        self.tracer = tracer
        self.server_host = server_host
        self.service_type = service_type
        self.operation_map = operation_map or {}
        self.capture_inputs = capture_inputs
        self.capture_input_max_len = capture_input_max_len
        self.enabled_paths = enabled_paths
        self.carrier_mode = carrier_mode

    def _operation_type(self, request: Request) -> str:
        key = f"{request.method.upper()}:{request.url.path}"
        return self.operation_map.get(key, f"{request.method.lower()}:
{request.url.path}")

    async def _try_get_json_body(self, request: Request) -> Optional[Dict[str,
Any]]:
        try:
            raw = await request.body()
            if not raw:
                return None
            return json.loads(raw.decode("utf-8", errors="ignore"))
        except Exception:
            return None

    def _safe_preview(self, value: Any) -> str:
        try:
            s = value if isinstance(value, str) else json.dumps(value,
ensure_ascii=False)
        except Exception:
            s = str(value)
        return s[: self.capture_input_max_len]

    async def dispatch(self, request: Request, call_next: Callable):
        if self.enabled_paths and request.url.path not in self.enabled_paths:
            return await call_next(request)

        operation_type = self._operation_type(request)

        body_json = None
        if self.capture_inputs:
            body_json = await self._try_get_json_body(request)

        span_name = f"genai.http {request.method.upper()} {request.url.path}"
```

```
        telemetry.setup_init_telemetry()
        tracer = telemetry.tracer
        server_host = telemetry.server_host

        span = tracer.start_span(span_name)
        span_context = set_span_in_context(span)
        token = attach(span_context)

        try:
            span.set_attribute("server.host", server_host)
            span.set_attribute("service.type", self.service_type)
            span.set_attribute("gen_ai.operation.name", operation_type)
            span.set_attribute("http.method", request.method)
            span.set_attribute("http.route", request.url.path)

            if body_json:
                span.set_attribute("gen_ai.request.body_preview",
self._safe_preview(body_json))
                if isinstance(body_json, dict) and body_json.get("session_id"):
                    span.set_attribute("gen_ai.conversation.id",
str(body_json.get("session_id")))

            response: Response = await call_next(request)

            if not isinstance(response, StreamingResponse):
                span.set_attribute("http.status_code", getattr(response,
"status_code", 200))
                span.set_status(Status(StatusCode.OK))
                span.end()
                return response

            original_iter = response.body_iterator

            async def wrapped_iter():
                inner_token = attach(span_context)
                try:
                    async for chunk in original_iter:
                        yield chunk
                    span.set_attribute("http.status_code", getattr(response,
"status_code", 200))
                    span.set_status(Status(StatusCode.OK))
                except Exception as e:
                    span.record_exception(e)
                    span.set_status(Status(StatusCode.ERROR, str(e)))
                    raise
                finally:
                    detach(inner_token)
                    span.end()

            response.body_iterator = wrapped_iter()
            return response

        except Exception as e:
            span.record_exception(e)
```

```
            span.set_status(Status(StatusCode.ERROR, str(e)))
            span.end()
            raise
        finally:
            detach(token)
```

---

2. Registrar el Middleware en `main.py`

En el archivo principal (`main.py`), añade el Middleware configurando las rutas objetivo.

```
import uvicorn
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

# ☑ 1. Importar el Middleware y configuración de telemetría
from app.presentation.api.genai_middleware.genai_sse_middleware import
GenAITrackingSSEMiddleware
from app import telemetry_singleton as telemetry

def create_app():
    app = FastAPI()

    # ☑ 2. Inicializar telemetría
    telemetry.setup_init_telemetry()
    tracer = telemetry.tracer
    server_host = telemetry.server_host

    # ☑ 3. Mapear rutas e identificar las operaciones
    OPERATION_MAP = {
        "POST:/api/chat": "agent-route-post",
        "POST:/api/chat/stream": "agent-route-post-stream",
        "POST:/api/sessions": "create-session-post",
    }
    ENABLED_PATHS = {"/api/chat", "/api/chat/stream", "/api/sessions"}

    # ☑ 4. Agregar el middleware a la App
    app.add_middleware(
        GenAITrackingSSEMiddleware,
        tracer=tracer,
        server_host=server_host,
        service_type="credicorp-dispatcher-middleware",
        operation_map=OPERATION_MAP,
        capture_inputs=True,
        enabled_paths=ENABLED_PATHS,
        carrier_mode="fallback"
    )

    # ... Configuración de rutas (routers) ...

    return app
```

```python
if __name__ == "__main__":
    app = create_app()
    uvicorn.run(app, host="0.0.0.0", port=8080)
```

Con esta configuración, toda solicitud a las rutas provistas será monitoreada de forma automática, permitiéndote registrar métricas y operaciones sin añadir código extra a los controladores.